

IMC PROSPERITY

Complete Competitive Analysis & Strategy Guide

Covering Rounds 1–5 of Prosperity 1 (2023), Prosperity 2 (2024), and Prosperity 3 (2025)

Compiled from top team writeups, GitHub repositories, and community insights

Prepared for IMC Prosperity 4 Preparation

INTRODUCTION: What is IMC Prosperity?

IMC Prosperity is a global algorithmic trading competition hosted by IMC Trading, one of the world's largest market-making firms. It runs for 15 days (typically), spread across five rounds, and is open to students worldwide with 7,000–12,000+ teams competing. The competition perfectly simulates real-world trading challenges in a gamified environment, making it ideal for learning quantitative finance, market microstructure, and systematic trading strategy.

Each round has two components: an Algorithmic Trading challenge (write a Python trading algorithm) and a Manual Trading challenge (solve a quantitative puzzle for maximum profit). The algorithmic profit typically dominates the leaderboard, but manual rounds can differentiate close competitors.

The Trading Environment

Your algorithm receives a `TradingState` object each timestep (~100ms intervals) containing:

- Order book (best bids/asks with volumes) for each product
- Your current position in each product
- Market trades that occurred in the previous tick
- Historical data you choose to store in 'traderData' (persisted as a string)

You can submit Orders (buy or sell at a price), and there are position limits per product (e.g., 50 units). The simulation processes your orders alongside bot orders, and you are neither first nor last — the sequence is: some deep-liquidity makers → occasional takers → your bot → more takers. This means speed/latency doesn't matter; you always have full book info before deciding.

The Key Insight About Prosperity's Market Structure

Critical Structural Insight: Prosperity markets use a SNAPSHOT model: every timestep, all previous orders are cancelled, and a fresh book appears. You cannot cancel or modify intra-tick. This means HFT-style order racing is irrelevant — the game is about correctly modeling fair value and being the best passive or aggressive participant every tick.

Recurring Trading Concepts Across All Rounds

The following concepts recur across all three Prosperity competitions — understanding them deeply is the foundation for competing well:

1. Market Making

Post passive buy orders below fair value and passive sell orders above fair value, profiting from the bid-ask spread. The key tension: wider quotes = more profit per fill but fewer fills; narrower quotes = more fills but less margin. The optimal approach is to observe where other bots are posting and either undercut/overbid them by 1 unit while maintaining positive edge. Inventory risk (holding too much of a position) must be managed by adjusting quote skew.

2. Statistical Arbitrage (Pair Trading / ETF Arb)

When two products have a known mathematical relationship (e.g., a basket = sum of components), any deviation from that fair relationship is tradeable. You trade the spread: buy the cheap side, sell the expensive side, and profit when the spread reverts. This appeared in P1 (Coconuts/Pina Coladas), P2 (Gift Basket), and P3 (Picnic Baskets).

3. Mean Reversion

Some prices deviate from a moving average and tend to revert. The strategy is to measure deviation (z-score), and buy when below the mean, sell when above. This requires a stable, estimable 'true value' — either a known constant, a spread, or a synthetic fair price.

4. Directional / Momentum Trading

When prices trend (persistently move in one direction), a momentum strategy buys into upward moves and sells into downward moves. This is riskier because it requires predicting direction. It worked in P1 for Berries (known seasonal curve) and Baguettes.

5. Options Pricing (Black-Scholes)

In P2 Round 4 and P3 Round 3, option-like products were introduced. Black-Scholes gives a fair value for a European call option given: underlying price S , strike K , time to expiry T , risk-free rate r , and implied volatility σ . Trading happens when the market price diverges significantly from the model price. Delta hedging (holding $-\Delta$ units of underlying per option) neutralises directional risk.

6. Locational / Conversion Arbitrage

In P2 Round 2 and P3 Round 4, products can be converted between 'locations' at a cost (transport fees, tariffs). If the conversion price + cost < market price elsewhere, there's arbitrage. The key is computing the effective all-in cost of moving a unit from one market to another.

7. Trader Signals (Informed Trader Exploitation)

In P1 Round 5 and P2 Round 5, anonymized trade data becomes de-anonymized. Certain bots (Olivia, Vladimir, Remy, Rihanna, Vinnie) trade with predictable patterns. If you observe that 'Olivia buys' reliably predicts price increases, you can piggyback on her signal.

PART I: IMC PROSPERITY 1 (2023)

Prosperity 1 was the inaugural edition of the competition, with approximately 7,007 teams. The winner was a European team. Stanford Cardinal (Shubham Anand Jain, Konstantin Miagkov, Parth Dodhia) finished 2nd overall. The competition introduced the core framework that all subsequent editions built upon.

Round	New Products	Theme
Tutorial/R1	Pearls, Bananas	Market Making + Regression Forecasting
Round 2	Coconuts, Pina Coladas	Pair Trading / Stat Arb
Round 3	Berries, Diving Gear	Seasonal Curves + External Signals
Round 4	Picnic Basket, Dip, Baguette, Ukulele	Basket / ETF Arbitrage
Round 5	All above + Trader IDs	Informed Trader Signals

ROUND 1 — Market Making on Pearls & Bananas

Algorithmic Challenge

Products Introduced

- PEARLS — Extremely stable, true value fixed at 10,000 seashells. Occasional bots trade irrationally at prices like 10,004, 10,005, 9,995, 9,996.
- BANANAS — Volatile, price changes from tick to tick. No fixed true value. Price follows a somewhat linear trend within a day.

Pearls: The Perfect Market-Making Problem

Pearls are the canonical 'easy' market-making product, appearing in every Prosperity edition under different names (Amethysts in P2, Rainforest Resin in P3). The true value is always 10,000 and never changes. Bots will occasionally submit market orders at 9,995–9,998 (ask) or 10,002–10,005 (bid), creating profitable opportunities for a market maker.

Strategy — Pearls: Two-part strategy: (1) TAKE: Aggressively buy any ask below 10,000 and sell any bid above 10,000. (2) MAKE: Post passive buy orders at 9,998 or 9,999 and passive sell orders at 10,001 or 10,002, hoping bots fill them. Inventory management: if position gets too long or short, close at 10,000 to free capacity.

Why it works: The true value is known with certainty. Any price above 10,000 is an overvalued ask — you sell into it. Any price below 10,000 is an undervalued bid — you buy from it. The only skill is in optimizing your quote placement to maximize fills while maintaining positive edge.

Python Pattern — Pearls Market Making

```
def trade_pearls(state, orders):
    TRUE_VALUE = 10000
    LIMIT = 20 # position limit
    pos = state.position.get("PEARLS", 0)
    book = state.order_depths["PEARLS"] # Step 1: Take favorable liquidity
    for ask, vol in sorted(book.sell_orders.items()):
        if ask < TRUE_VALUE and pos < LIMIT:
            qty = min(-vol, LIMIT - pos)
            orders.append(Order("PEARLS", ask, qty))
            pos += qty
    for bid, vol in sorted(book.buy_orders.items(), reverse=True):
        if bid > TRUE_VALUE and pos > -LIMIT:
            qty = max(-vol, -LIMIT - pos)
            orders.append(Order("PEARLS", bid, qty))
            pos += qty # Step 2: Post passive quotes with edge
    if pos < LIMIT:
        orders.append(Order("PEARLS", TRUE_VALUE - 2, LIMIT - pos)) # buy quote
    if pos > -LIMIT:
        orders.append(Order("PEARLS", TRUE_VALUE + 2, -LIMIT - pos)) # sell quote
```

Bananas: Regression-Based Fair Value

Bananas introduced the challenge of trading a product with an unknown, moving true value. The price was not stable but followed somewhat predictable short-term trends. Key observations from top teams:

- Prices followed a linear-ish trend across days — day N's regression coefficients were similar to day N-1's.
- Short-term (last 3–5 ticks) linear regression on prices predicted the next price reasonably well.
- The popular bid and popular ask (prices with maximum order volume) were good estimates of fair value.

Strategy — Bananas (Stanford #2 approach): Run a linear regression over the last few timesteps of prices to predict the next price. Use this prediction as the fair value: buy aggressively (take) when market price is below prediction, and post passive orders (make) around the predicted price. Additionally, market-make around the predicted price with a spread of 2-4 units.

The concept of the 'popular mid price' (the midpoint between the price level with the most buy volume and the price level with the most sell volume) was found to be a robust, simple fair value estimator that worked without regression. This technique reappeared across all Prosperity editions.

Key Algorithmic Lessons from P1 Round 1

- Always implement market-making on stable products first — it's free money with almost no risk.
- For volatile products, estimate fair value using either regression or the 'popular mid' heuristic.
- Build a backtester early — Stanford's final round success was partly due to having a backtester from Round 4.
- Avoid overcomplicating: complex LR-on-the-fly approaches had more bugs and lower performance than simple methods.

Manual Challenge — Round 1

Problem Statement

You are given a fixed pool of fish, each with a reserve price drawn uniformly from [900, 1000]. You submit two bids: a low bid and a high bid. A fish accepts your low bid if its reserve price is at or below your low bid. If rejected by the low bid, it accepts your high bid if the reserve price is at or below your high bid. All gear bought can be sold at exactly 1000 seashells. Maximize your total profit across all fish.

Mathematical Setup

Let the two bids be L (low) and H (high), where $900 \leq L < H \leq 1000$. The fraction of fish accepting L is $(L - 900)/100$. Those accepting L give profit $(1000 - L)$ each. The fraction accepting H (but not L) is $(H - L)/100$, giving profit $(1000 - H)$ each.

Total profit $\propto (L-900)(1000-L) + (H-L)(1000-H)$. Taking partial derivatives and setting to zero yields the optimal bids. The closed-form solution, found independently by Edgar Maddocks and confirmed by Monte Carlo simulations by jmerle, is:

Optimal Solution — P1 Round 1 Manual: Low bid $L^* = 952$, High bid $H^* = 978$. These maximize the combined expected profit. Monte Carlo simulation with 10M fish confirms this. Teams that ran this simulation correctly all converged to 952/978.

Teams that used simulation (even crude ones) arrived at the correct answer. The math is similar to a monopolist pricing problem — classic in microeconomics — where you're maximizing expected revenue from consumers with uniformly distributed willingness-to-pay.

ROUND 2 — Pair Trading: Coconuts & Pina Coladas

Algorithmic Challenge

Products Introduced

- COCONUTS — The underlying commodity. Position limit: 300.
- PINA_COLADAS — A composite product with known composition: 1 Pina Colada = 15/8 Coconuts (equivalently, 8 Pina Coladas = 15 Coconuts). Position limit: 150.

The Pair Trading Opportunity

The fundamental relationship is: $\text{Fair Price}(\text{PINA_COLADAS}) = (15/8) \times \text{Price}(\text{COCONUTS})$. In practice, this ratio held most of the time, but small deviations appeared. When the spread (Pina Colada price - $15/8 \times$ Coconut price) diverged from its historical mean, it was a strong signal to trade: sell the relatively expensive one and buy the relatively cheap one, expecting the spread to converge.

Strategy — Coconut/Pina Colada Pair Trading: Compute spread = $\text{PINA_COLADAS_mid} - (15/8) \times \text{COCONUTS_mid}$. Track a rolling mean and standard deviation of this spread. When z-score = $(\text{spread} - \text{mean}) / \text{std}$ exceeds a threshold (e.g., +1.5): sell PINA_COLADAS and buy COCONUTS. When z-score < threshold (e.g., -1.5): buy PINA_COLADAS and sell COCONUTS. Exit when z-score returns toward 0.

Stanford's team implemented this and found it worked significantly better than momentum-based trading on these products. The key parameters to tune are: (1) the lookback window for the rolling mean/std, (2) the entry z-score threshold, (3) the exit threshold.

What Went Wrong — The Cautionary Tale

Stanford dropped from 9th to 926th place in this round due to a bug in their algorithm. The lesson: even a well-conceived strategy can destroy your ranking if implementation is incorrect. The AWS Lambda errors that plagued the competition also affected score accuracy in this round. Always test your algorithm thoroughly on provided historical data before submission.

Key Algorithmic Lessons from P1 Round 2

- When two products have a known mathematical relationship, always implement stat-arb — it has provable positive EV.
- Test on multiple days of historical data, not just one, to avoid overfitting entry/exit thresholds.
- The ratio 15/8 was given in the problem description — always read the problem carefully for embedded fair value formulas.
- Bugs kill rankings more than suboptimal parameters. Implement simple, correct code first.

Manual Challenge — Round 2

Problem Statement

You are given a fish population where each fish has a reserve price. You must guess the median reserve price of the bottom 50% of fish (by reserve price). The closer your guess is to the true median, the more money you earn.

Approach

Teams ran simulations to estimate what the median of the bottom half of a distribution would be. Stanford submitted 9,777 (the true answer was a median of 9,685 for the bottom 50%). The scoring penalized distance from the true value, so simulation-based approaches or careful statistical reasoning were rewarded.

ROUND 3 — Seasonal Curves & External Signals

Algorithmic Challenge

Products Introduced

- BERRIES — Price follows a known seasonal intraday curve. Price rises during the day, peaks mid-day, then falls.
- DIVING_GEAR — Price is driven by Dolphin sightings. Large changes in sightings (± 5 or more) signal genuine price moves.

Berries: Exploiting a Known Price Curve

Berries had a characteristic intraday pattern that was consistent across trading days. The key insight was that this curve was provided (or observable from data) — it was not hidden. The strategy was essentially trend-following aligned with the known curve:

- Buy aggressively at the known trough (timestamp $\sim 350k$)
- Sell aggressively at the known peak (timestamp $\sim 500k$)
- After the peak, optionally buy or sell depending on the overall day's trend for the decline phase

Strategy — Berries (Stanford approach): Hardcode the key timestamps: BUY at $t=350k$, SELL at $t=500k$. For the declining phase, infer direction from day-level trend. Buy at limit; sell at limit. This simple approach captured the bulk of the available profit without requiring dynamic prediction.

The broader principle: when a price curve is deterministic or near-deterministic, the optimal strategy is to trade aggressively at the extremes — don't try to capture micro-movements, just bet big at the known turning points.

Diving Gear: Signal Filtering

Diving Gear's price was correlated with Dolphin sightings data provided each tick. However, not all changes in sightings were meaningful — small fluctuations (± 1 or ± 2) were noise, while large changes (± 5 or more) indicated a genuine regime shift.

Strategy — Diving Gear (Stanford approach): Only act on large Dolphin sighting changes (absolute delta ≥ 5 from the previous reading). When dolphin sightings spike up strongly, buy Diving Gear. When they spike down strongly, sell. Ignore small changes as noise. This filtering prevented whipsaw losses from reacting to random fluctuations.

Key Algorithmic Lessons from P1 Round 3

- When an external signal drives price, always implement a noise filter — distinguish signal from noise using a threshold.
- Known price curves are gold: hardcode key buy/sell timestamps rather than trying to dynamically detect the peak.
- Stanford had the highest round score in this round, moving from 926th to 60th place — showing how quickly a good round can recover rankings.

Manual Challenge — Round 3

Problem Statement

You must guess a specific price (near 9,969) to maximize your profit. The scoring rewards proximity to the true optimal value. Stanford entered 9,959 (within 10 of optimal 9,969), which was excellent. The lesson: sometimes the manual challenge is about estimation and 'being close' rather than finding an exact answer.

ROUND 4 — Basket / ETF Arbitrage

Algorithmic Challenge

Products Introduced

- PICNIC_BASKET — A composite product containing: $4 \times \text{DIP} + 2 \times \text{BAGUETTE} + 1 \times \text{UKULELE} + 375$ premium
- DIP, BAGUETTE, UKULELE — The individual component products

The ETF Arbitrage Framework

The Picnic Basket is analogous to an ETF: it's a bundle of underlying assets trading as a single instrument. The basket's 'fair' price is the sum of its components plus an expected premium (in this case, approximately 375 seashells). When the basket trades above or below this synthetic price, there's an arbitrage opportunity.

Define the $\text{SPREAD} = \text{PICNIC_BASKET_mid} - (4 \times \text{DIP_mid} + 2 \times \text{BAGUETTE_mid} + \text{UKULELE_mid} + 375)$. Historically, this spread oscillates around 0. When it deviates significantly:

- Spread $\gg 0$: Basket is overpriced relative to components. SELL basket, BUY components.
- Spread $\ll 0$: Basket is underpriced relative to components. BUY basket, SELL components.

Strategy — Picnic Basket (Stanford approach): Compute spread = PICNIC_BASKET - (4×DIP + 2×BAGUETTE + UKULELE + 375). Track rolling z-score. Trade basket exclusively on this signal. Stanford found that focusing only on the basket (not trying to simultaneously arb all components) gave the best risk-adjusted return, as the basket always had 'heavily positive PnL'.

What to Trade: Basket Only vs. Full Hedge

There's a debate in every ETF-arb round about whether to trade only the basket or to hedge with components. Stanford's finding was instructive: trading only the basket (using the component prices as a signal but not trading them) was more profitable and less risky than attempting a full hedge. This is because:

- Components have their own position limits, so a full hedge might hit limits on DIP or BAGUETTE before you can complete the trade.
- The basket's PnL was consistently positive, suggesting the 'informed' bots were systematically trading basket at off-fair prices.
- Attempting to leg into spreads (trading legs sequentially) introduces execution risk.

Building a Backtester

Stanford built a backtester in Round 4 that ran their algorithm's run() function directly against historical data. This was a critical infrastructure investment that paid dividends in Round 5. A backtester should:

- Feed historical order book data tick by tick to your algorithm
- Simulate order matching at the prices your algorithm submits
- Track PnL and position across the simulation
- Allow parameter sweeping (grid search) over strategy parameters

Manual Challenge — Round 4

Problem Statement

You must allocate capital across multiple stocks. If stock i moves $x_i\%$ and you invest $y_i\%$ of your capital, your payoff from that stock is $75 \times x_i \times y_i - 90 \times y_i^2$. The total payoff is the sum across all stocks. Maximize total payoff.

Mathematical Solution

For a single stock, the optimal investment fraction y^* maximizes $75xy - 90y^2$. Taking the derivative with respect to y : $75x - 180y = 0$, giving $y^* = 75x/180 = 5x/12 \approx 2.5x/6$. So you should invest a fraction proportional to the expected move size.

Strategy — P1 R4 Manual: Estimate how much each stock will move (x_i). Set your investment fraction $y_i = 2.5 \times x_i / 6$ for each stock. Sum your y_i fractions — they may sum to more or less than 100%; normalize if needed. Stanford invested approximately 100% of capital. Best scores around 115k, Stanford got 75k (a gap due to imprecise x_i estimates).

ROUND 5 — Trader Signals & Final Optimization

Algorithmic Challenge

New Information: Trader Identities

Round 5 revealed the identities of market participants. The key insight was that some traders (named bots) had consistent predictive patterns. Stanford identified Olivia as an informed trader:

- Olivia bought Ukuleles at the lowest prices and sold at the highest — she was essentially a perfect signal for Ukulele direction.
- Trading exclusively when Olivia traded (following her side) was highly profitable for Ukuleles.
- For Berries, Olivia's signal augmented the existing seasonal strategy.

Portfolio-Level Decisions

Stanford's final strategy reflected disciplined portfolio thinking:

- Trade ONLY the Picnic Basket on the spread signal — not DIP, BAGUETTE, or UKULELE directly (except via Olivia).
- Trade ONLY PINA_COLADAS on the coconut spread signal — not COCONUTS directly.
- Never trade DIP, BAGUETTE, or COCONUTS.
- Use momentum on Baguettes only if parameters are stable across validation sets (they weren't, so this was dropped).

Key Risk Management Principle from Stanford: 'Be rigorous about not overfitting, and avoid complicated models due to the lack of test data.' This is the most important algorithmic trading principle for Prosperity: simple, robust strategies beat complex overfitted ones every time, especially with limited historical data.

Manual Challenge — Round 5

Problem Statement

Not detailed in available sources, but implied to involve stock market news with directional signals similar to P2 Round 5's 'Iceberg news' challenge.

PART II: IMC PROSPERITY 2 (2024)

Prosperity 2 had approximately 9,000+ teams. The competition re-used the structural format of Prosperity 1 but introduced new products including Orchids (conversion arbitrage), a refreshed Gift Basket (Chocolate, Strawberries, Roses), and Coconut Coupons (options). Crucially, the Coconut data in P2 was nearly identical to P1's Coconut data — teams that noticed this could exploit it for massive PnL. The top 4 teams in the final leaderboard reportedly all exploited this cross-year data similarity.

Jasper van Merle (jmerle, solo team 'camel_case') finished 9th overall — one of the most impressive solo performances, especially notable for his open-sourced tools (visualizer, backtester, submitter) that helped many other participants.

Round	New Products	Theme
Tutorial/R1	Amethysts, Starfruit	Market Making (P1 Pearls/Bananas equiv.)
Round 2	Orchids	Conversion / Locational Arbitrage
Round 3	Chocolate, Strawberries, Roses, Gift Basket	ETF Statistical Arbitrage
Round 4	Coconuts, Coconut Coupons	Options Pricing (Black-Scholes)
Round 5	All + Trader IDs revealed	Informed Trader Signals

ROUND 1 — Market Making on Amethysts & Starfruit

Algorithmic Challenge

Products: Amethysts (= P1 Pearls) and Starfruit (= P1 Bananas)

Amethysts are stable at 10,000. Starfruit fluctuate. The market-making logic is identical to P1 Round 1 — if you did Prosperity 1, you already know this.

The Popular Bid/Ask Heuristic (Jmerle's Key Insight)

Rather than using a simple midprice (average of best bid and ask), jmerle used the 'popular bid' and 'popular ask' — the bid price and ask price that had the highest volume in the order book. The midpoint of these two prices proved to be a better estimate of fair value than the arithmetic midprice, because it reflects where the majority of market participants think the price is.

Strategy — Starfruit (jmerle 9th place): Fair value = midpoint between popular_bid (bid price with max volume) and popular_ask (ask price with max volume). Post buy orders at fair_value - 1 and fair_value - 2, sell orders at fair_value + 1 and fair_value + 2. Implement 'soft' and 'hard' liquidation: at position 80-90% of limit, start accepting worse prices; at limit, close at any cost.

Soft/Hard Liquidation

A critical risk management technique: when your position is near its limit, you can no longer market-make symmetrically (you can't buy more if you're at the long limit). The solution is tiered liquidation:

- Soft liquidation: At 80% of limit, shift your quote center — post fewer passive orders on the side that would worsen your position.
- Hard liquidation: At the limit, actively submit market orders to reduce position to a safer level, even at unfavorable prices.
- This prevents getting stuck at the limit and missing profitable trades on the other side.

jmerle's Round 1 Performance

210th overall, 265th algo, 349th manual. The algorithm was essentially unchanged from his tutorial — proving that in Round 1, the correct market-making setup is sufficient.

Manual Challenge — Round 1 (Fish Gear Auction)

Problem Statement

Identical in structure to P1 Round 1 with slight changes to parameters. Fish have reserve prices drawn uniformly from [900, 1000]. You submit two bids L and H ($L < H$). Any fish with reserve price $\leq L$ accepts your low bid; any fish with reserve price in $(L, H]$ accepts your high bid. All accepted gear sells for 1000. Maximize total profit.

Solution

Optimal Bids — P2 R1 Manual: $L^* = 952$, $H^* = 978$. This is the same solution as P1. Multiple teams confirmed this via closed-form calculus (Edgar Maddocks's writeup) and Monte Carlo simulation (jmerle ran 10M samples). Teams that submitted 952/978 tied for ~4th in the manual round.

The formula is derived by maximizing total profit $= (L-900)/100 \times (1000-L) + (H-L)/100 \times (1000-H)$. Setting $\partial/\partial L = 0$ and $\partial/\partial H = 0$ gives a system of equations whose solution is $L=952$, $H=978$.

ROUND 2 — Orchids: Conversion Arbitrage

Algorithmic Challenge

The Conversion Mechanism

Orchids were the first product with a cross-market trading mechanism. There were two 'locations' (North and South archipelagos). You could observe the Orchid price in both markets and convert between them at a cost. If the price difference between markets exceeded the conversion cost, arbitrage was possible.

The conversion mechanism was: hold a long position in Market A, convert it to an equivalent position in Market B by paying a conversion fee. If $\text{Market B price} - \text{Market A price} > \text{conversion fee}$, you profit from the conversion.

The Key Strategy (jmerle, 6th place algo this round)

jmerle learned from Discord hints the core trick: continuously short sell Orchids to the position limit at a price where immediate profitable conversion back to flat was possible. Then convert to 0 and immediately go short again in the same iteration. This generated consistent profit by exploiting the conversion spread.

Strategy — Orchids (jmerle approach): Each tick: (1) Check if short selling at current ask price and converting to 0 is profitable (i.e., $\text{short_price} - \text{conversion_cost} > 0$). (2) If yes, sell to the short limit, converting immediately to 0. (3) Repeat. The trick was realizing you can: sell short → convert to flat → sell short again, all within ONE timestep. This doubled the profit per tick vs. a naive single-trade approach.

This 'double dip' insight — discovering that two operations could be sequenced in the same tick — was the key to unlocking 100k+ profits this round. It exemplifies the importance of carefully reading the competition's API documentation and experimenting with order sequencing.

Why Many Teams Struggled

Teams that tried to model Orchid prices using environmental variables (sunlight, humidity, etc. provided in the data) couldn't make it work reliably. The conversion arbitrage was the primary mechanism, but finding it required understanding the conversion API deeply rather than treating Orchids as just another price-prediction problem.

Manual Challenge — Round 2 (Currency Conversion Chain)

Problem Statement

You are given a set of exchange rates between different items (seashells, pizza slices, wasabi roots, etc.). Using a maximum of N conversions, find the sequence of conversions that maximizes your final seashell count starting from 1 seashell.

Solution

This is a classic graph problem: find the path of length $\leq N$ in a directed weighted graph (exchange rate graph) that maximizes the product of edge weights. A simple dynamic programming or DFS/BFS approach works. jmerle computed all profitable paths and found: seashells $\rightarrow$ pizza slices $\rightarrow$ wasabi roots $\rightarrow$ seashells $\rightarrow$ pizza slices $\rightarrow$ seashells, giving 13.9% total profit (113,938.8 seashells from an initial 100,000).

Algorithm — P2 R2 Manual: Build an exchange rate graph where each edge weight is the conversion factor. Run DFS/BFS up to the max conversion depth. Multiply conversion rates along each path. The path with the highest product is the optimal sequence. Always check if you end in seashells (the base currency) for a valid final state.

ROUND 3 — Gift Basket ETF Arbitrage

Algorithmic Challenge

Products: Chocolate, Strawberries, Roses, Gift Basket

The Gift Basket = 4 CHOCOLATE + 6 STRAWBERRIES + 1 ROSES. This is analogous to P1's Picnic Basket but with different components and no stated premium. The basket's fair value can be computed as a weighted sum of component mid-prices.

The Spread-Based Arbitrage

Define: $\text{SPREAD} = \text{GIFT_BASKET_mid} - (4 \times \text{CHOCOLATE_mid} + 6 \times \text{STRAWBERRIES_mid} + \text{ROSES_mid})$. Track the rolling mean and standard deviation of this spread. When the spread deviates significantly from its mean ($z\text{-score} > \text{threshold}$), trade the basket:

- $z\text{-score} \gg 0$ (basket expensive): Sell Gift Basket, optionally buy components.
- $z\text{-score} \ll 0$ (basket cheap): Buy Gift Basket, optionally sell components.

Strategy — Gift Basket (jmerle, 11th algo this round): Set entry threshold so that when $|z\text{-score}| > 1.0$, go 100% long or short on the basket. Use grid search over the threshold parameter (ranges from 0.5 to 2.0) on historical data. jmerle found trading basket alone was safer than component hedging — roses caused losses of 36k when included.

Why Trading Components Is Risky

jmerle's experience is instructive: when he tried hedging with all components, Roses caused 36k losses. This is because individual components may have their own dynamics that aren't purely driven by the basket relationship. The basket itself might revert while a component diverges for its own reasons (informed traders, other signals). The lesson: start simple (basket only), and only add component trades if backtesting shows robust improvement.

Key Insight: 'Basket Rerun' — Copy from P1 Round 4

jmerle noted this round was 'basically a copy-paste of last year's round 4.' Teams that participated in P1 had a significant head start. This cross-year pattern is critical to remember for Prosperity 4 — always study all previous years' rounds for patterns that may repeat.

Trader Signal: Vladimir/Remy on Chocolate, Rihanna on Roses

In Round 5 when trader IDs were revealed, the following signals were found (applicable retroactively):

- If Vladimir sells CHOCOLATE to Remy → go long on CHOCOLATE.
- If Remy sells CHOCOLATE to Vladimir → go short on CHOCOLATE.
- If Rihanna sells ROSES to Vinnie → go long on ROSES.
- If Vinnie sells ROSES to Rihanna → go short on ROSES.

Manual Challenge — Round 3 (Treasure Map Expedition)

Problem Statement

You are given a map with tiles, each with a multiplier M . The base reward per tile is 7,500 seashells. Your payout = $7,500 \times M / (\text{num_hunters_on_tile} + 1)$. You can search 1–3 tiles: 1st search is free, 2nd costs 25,000, 3rd costs 75,000. The number of hunters on a tile grows with the percentage of players choosing it (1% of all players = 1 additional hunter). Choose tiles to maximize expected personal payout.

Solution Approach

This is a Nash Equilibrium problem with incomplete information about other players' choices. Key steps:

- Assume all rational players choose tiles with the highest expected value (given others' choices).
- Compute EV for each tile assuming a distribution of player choices (the simplest assumption: uniform at first, then update based on observed behavior).
- The tile with the highest raw multiplier (e.g., 100×) will be overcrowded — its EV will be diluted. Medium-multiplier tiles with low initial hunters are often better.
- Use two expeditions if the 2nd expedition's expected gain exceeds 25,000. Use three if the 3rd clears 75,000.

Strategy — P2 R3 Manual (jmerle approach): Combine: (1) gut feeling about which tiles look undervalued, (2) mathematical simulation assuming rational play, (3) community data from a shared spreadsheet of reported picks. jmerle chose tiles at expeditions 83 and 87, earning 78,577 vs a maximum of 106,291. The game-theory optimum depends on predicting competitor behavior — there's no single 'correct' answer.

ROUND 4 — Options: Black-Scholes on Coconut Coupons

Algorithmic Challenge

Products: Coconuts (underlying) and Coconut Coupons (options)

Coconut Coupons are call option-like instruments on Coconuts. A call option gives the right (not obligation) to buy the underlying at a fixed price (strike price K) at expiry. The fair value of a European call option is given by the Black-Scholes formula:

$$C = S \times N(d1) - K \times e^{(-rT)} \times N(d2)$$

where $d1 = [\ln(S/K) + (r + \sigma^2/2)T] / (\sigma\sqrt{T})$, $d2 = d1 - \sigma\sqrt{T}$

Parameters: S = current Coconut price, K = strike of the Coupon, r = risk-free rate (often approximated as 0), T = time to expiry (in years, compute from timesteps remaining), σ = implied volatility (to be estimated from market prices), $N(\cdot)$ = standard normal CDF.

The Trading Strategy

Compute the Black-Scholes fair value of each Coupon each tick. Compare it to the actual market price of the Coupon:

- Coupon market price > BS fair value: Coupon is overpriced → SELL Coupons.
- Coupon market price < BS fair value: Coupon is underpriced → BUY Coupons.
- Optional: delta hedge by holding $-\Delta$ units of Coconuts per Coupon, where $\Delta = N(d1)$.

Strategy — Coconut Coupons (jmerle, 9th overall): Applied Black-Scholes with σ calibrated from market prices (implied volatility). Used grid search on σ to find parameters yielding ~420k profit in backtests. Earned 184k on Coupons in actual round. Stopped trading Coconuts directly (lost 77k from a directional bet on Coconuts that failed). Key lesson: bet on the derivative, not the underlying.

The Cross-Year Data Exploit

A major meta-insight from P2: the Coconut data in Prosperity 2 was nearly identical to the Coconut data from Prosperity 1. Teams that recognized this could use P1's historical data as P2's 'future' data, knowing exactly what prices would be. The top 4 teams all seem to have exploited this, earning multi-million PnL from Coconuts in Rounds 4–5. This underscores the importance of comparing new round data against all historical Prosperity datasets.

Manual Challenge — Round 4 (Fish Gear Auction — Average-Aware)

Problem Statement

Same structure as Round 1 with one crucial modification: for the second bid, the fish know the average of all participants' second bids. A fish accepts your second bid if it exceeds this average; if below, it accepts probabilistically based on the shortfall. The first bid works the same as before.

Solution

For the first bid, optimal is still 952 (unchanged). For the second bid, you now need to bid AT or ABOVE the population average to maximize fills. jmerle's reasoning: most teams would submit their Round 1 optimal (978) for the second bid, making the average approximately 978–980. To maximize fills, bid at or slightly above the expected average.

Optimal Bids — P2 R4 Manual: First bid: 960 (intentionally slightly higher than 952 to get more first-bid fills since the second bid's EV was expected to be lower). Second bid: 980. The actual average bid turned out to be 980, so submitting exactly 980 for the second bid earned maximum profit. jmerle earned 102,592 — close to the maximum.

ROUND 5 — Trader IDs Revealed: Signal Exploitation

Algorithmic Challenge

De-anonymized Trade Data

Round 5 provided trade history with trader names. jmerle identified four signals through visual analysis:

Trader A	Action	Trader B	Signal for
Vladimir	Sells CHOCOLATE to	Remy	Go LONG Chocolate

Remy	Sells CHOCOLATE to	Vladimir	Go SHORT Chocolate
Rihanna	Sells ROSES to	Vinnie	Go LONG Roses
Vinnie	Sells ROSES to	Rihanna	Go SHORT Roses

These signals were found by: (1) visual inspection of trading charts with trader IDs color-coded, and (2) a grid search of 3,600 backtests to find any signals missed visually. The grid search found nothing new beyond the four above.

Final Algorithm — P2 Round 5 (jmerle)

- AMETHYSTS: Market-making at 10,000 (unchanged from Round 1)
- STARFRUIT: Market-making at popular mid (unchanged)
- ORCHIDS: Conversion arbitrage (unchanged from Round 2)
- CHOCOLATE: Directional trading on Vladimir/Remy signal
- STRAWBERRIES: Directional with overfit parameters from Round 3 data
- ROSES: Directional trading on Rihanna/Vinnie signal
- GIFT_BASKET: Directional with overfit parameters from Round 3
- COCONUT_COUPON: Black-Scholes arbitrage (from Round 4)
- COCONUT: Nothing (no reliable strategy found)

Manual Challenge — Round 5 (Iceberg News / Stock Selection)

Problem Statement

You are given a fictional news newspaper ('The Iceberg') with articles about various fictional companies. Based on these news items, allocate your 1,000,000 seashell portfolio across long/short positions in these companies' stocks, earning profit proportional to the actual price move $\times$ your allocation.

Solution Framework

The challenge requires qualitative interpretation of news catalysts and quantitative position sizing:

- Identify clear positive catalysts $\rightarrow$ Long position. Examples: product launch success, earnings beat, regulatory approval.
- Identify clear negative catalysts $\rightarrow$ Short position. Examples: recall, lawsuit, earnings miss, regulatory ban.
- Size positions by conviction: Strong signal = 15-25% allocation; Moderate = 5-10%; Weak = 1-3%.
- Diversify: don't put all capital in one position even if the signal is very strong.
- jmerle earned 115k (best was 149k, optimal was 157k). The gap was due to uncertain directional calls on some stocks.

jmerle's P2 R5 Manual Portfolio: Electric blankets (Sell), sleds (Sell), ice sculptures (Buy), PS6 (Buy), moustache serum (Sell), icicle earrings (Buy) with 10% weight. Did NOT trade refrigerators, lava lamps, hot chocolate (too uncertain). Split 90% equally between confident positions, 10% on less confident.

PART III: IMC PROSPERITY 3 (2025)

Prosperity 3 had 12,000+ teams. Frankfurt Hedgehogs (Timo Diehm, Arne Witt, Marvin Schuster) finished 2nd globally with a score of 1,433,876 seashells. The competition was won by Heisenberg. Prosperity 3 introduced the most sophisticated product set yet: options (Volcanic Rock Vouchers with 5 strikes), ETF baskets with two basket products simultaneously, and Magnificent Macarons (locational arbitrage with environmental factors like sunlight index).

Round	New Products	Theme
Tutorial/R1	Rainforest Resin, Kelp, Squid Ink	Market Making (stable + random walk + polarizing asset)
Round 2	Croissants, Jams, Djembes, Picnic Basket 1&2	ETF Statistical Arbitrage (two baskets)
Round 3	Volcanic Rock, 5 Voucher strikes (9500–10500)	Options: IV smile, Black-Scholes, delta hedging
Round 4	Magnificent Macarons	Location Arbitrage with sunlight/environmental signals
Round 5	All products + Trader IDs	Signal exploitation + strategy refinement

ROUND 1 — Market Making: Resin, Kelp, and Squid Ink

Algorithmic Challenge

Understanding the Snapshot Market Structure (Critical for All Products)

Frankfurt Hedgehogs' key insight about Prosperity's trading engine, directly applicable to every round: the simulation processes orders sequentially each tick — deep-liquidity makers first, then occasional takers, then your bot, then more takers. This means:

- You always see a complete order book snapshot before deciding.
- Your orders from the previous tick are automatically cancelled — you start fresh each tick.
- There's no advantage to being 'fast.' You simply cannot be sniped between ticks.
- You can simultaneously take existing orders AND post new passive orders in the same tick.

The Wall Mid — A Better Fair Value Estimate

Frankfurt Hedgehogs introduced the concept of 'Wall Mid' as a proxy for the true price. Rather than using the simple arithmetic mid ($\text{best_bid} + \text{best_ask} / 2$), the Wall Mid uses the bid and ask prices where the largest volume 'walls' exist. This is similar to Jmerle's 'popular mid' concept.

Wall Mid vs. Simple Mid: Simple mid = $(\text{best_bid} + \text{best_ask}) / 2$ — works but can be noisy when best quotes are small. Wall Mid = midpoint between the bid level with highest cumulative volume and the ask level with highest cumulative volume — more robust because it reflects where most liquidity is anchored. Frankfurt Hedgehogs used this for all products.

Rainforest Resin — The Perfect Market Maker

Identical to P1 Pearls and P2 Amethysts: true price permanently fixed at 10,000. The Frankfurt Hedgehogs strategy:

- Step 1 (TAKE): Immediately buy any ask below 10,000 and sell any bid above 10,000. Do this first, every tick.

- Step 2 (MAKE): Post passive buy orders one unit better than the best existing bid (while maintaining positive edge); post passive sell orders one unit better than the best existing ask.
- Step 3 (INVENTORY): If position is too large, flatten at 10,000 to free capacity for the next profitable opportunity.

This strategy contributed approximately 39,000 seashells per round consistently. It's the easiest 39k you'll ever make — don't overcomplicate it.

Kelp — Market Making on a Random Walk

Kelp's price follows a slow random walk. Despite this, the optimal strategy is nearly identical to Resin: treat the current Wall Mid as fair value and market-make around it. Key insights from Frankfurt Hedgehogs:

- The future price of Kelp is essentially unpredictable from the order book alone.
- There is a minor mean-reversion tendency because the true price is a float but quotes are integers — but this effect is too small to justify complexity.
- Simply quoting around the current Wall Mid, taking favorable trades, and managing inventory is sufficient.
- Teams that wasted time building linear regression or ML models for Kelp underperformed those who used simple market-making.

Squid Ink — The Polarizing Asset

Squid Ink was the most controversial product in P3. Many teams lost money on it; some made significant profit. The approaches attempted included:

- Z-score mean reversion (buy when price drops below rolling mean by n standard deviations)
- Statistical arbitrage relative to Kelp or Resin
- Volatility recoil (trade the return to lower volatility after a spike)
- Market-making with wider spreads to capture volatility premium

Martin Oravec (73rd globally) found that none of his Squid Ink strategies worked well live and chose not to trade it after Round 1. jmerle (25th) initially lost 17,873 on it but gained 18,268 after a rerun. Frankfurt Hedgehogs' final algorithm appears to have traded Squid Ink profitably. The correct approach (revealed in post-competition discussions) involved recognizing a specific momentum or signal pattern, but this was not widely discovered during the competition.

Squid Ink Lesson: If you cannot find a consistently profitable strategy in backtesting, do NOT trade the product in the real round. The cost of a bad Squid Ink strategy (losing money) exceeds the opportunity cost of skipping it (0 profit). This is a risk management principle: asymmetric outcomes favor inaction when uncertain.

Manual Challenge — Round 1 (FX Arbitrage)

Problem Statement

You are given a set of exchange rates between 4 currencies. Using at most 5 trades, find the sequence of conversions that maximizes profit from an initial 1 unit of the base currency. You keep any profit above 100% of the starting amount.

Solution

This is a classic arbitrage cycle detection problem on a directed weighted graph. The optimal approach:

- Model as a directed graph where each node is a currency and each edge weight is the conversion rate.
- Use BFS or DFS with depth ≤ 5 to enumerate all paths that start and end at the base currency.
- The maximum profit found was 8.9% ($1.089\times$ starting capital). Most teams found this exact answer since the solution space is small.

Algorithm — P3 R1 Manual: def find_best_path(rates, start, max_depth=5): BFS from 'start', multiplying exchange rates. Track running product for each path. Return path with maximum final value that ends at 'start'. This is $O(N^{\text{max_depth}})$ where N is number of currencies — trivially solvable for 4 currencies and depth 5.

ROUND 2 — ETF Statistical Arbitrage: Two Picnic Baskets

Algorithmic Challenge

Products Introduced

- CROISSANTS, JAMS, DJEMBES — Individual component products.
- PICNIC_BASKET1 = 6 CROISSANTS + 3 JAMS + 1 DJEMBE
- PICNIC_BASKET2 = 4 CROISSANTS + 2 JAMS

Basket Synthetic Fair Value

For each basket, compute its synthetic fair price from component mid-prices:

```
Fair(BASKET1) = 6 × mid(CROISSANT) + 3 × mid(JAM) + mid(DJEMBE)
Fair(BASKET2) = 4 × mid(CROISSANT) + 2 × mid(JAM)
```

Two-Way Stat Arb

Martin Oravec (73rd) implemented a 'two-way statistical arbitrage' that also priced components from baskets — not just baskets from components. This means:

- When Basket 1 is expensive relative to components: sell Basket 1, price components as fair based on Basket 1.
- When Basket 1 is cheap: buy Basket 1, price components as fair based on Basket 1 + Basket 2 information.
- Both baskets and components constantly pull each other back into alignment.

Strategy — Picnic Baskets (Frankfurt Hedgehogs / Oravec): Compute z-score of (Basket_mid - Synthetic_fair) over a rolling window. Entry signal at $|z| > \text{threshold}$ (tune via grid search, typically 1.0–1.5). Trade basket exclusively on this signal. For component pricing, cross-reference both baskets. Frankfurt Hedgehogs additionally exploited hardcoded patterns visible in their dashboard (basket premium has predictable intraday patterns).

Frankfurt Hedgehogs' 'Hardcoding' Insight

The Frankfurt Hedgehogs mention 'hardcoding in the first two rounds' in their FAQ. This refers to the fact that they studied their dashboard extensively and found that certain basket/component relationships had specific, repeatable patterns that could be exploited by hardcoding specific price levels or time-based triggers rather than pure dynamic estimation. This is data-driven overfitting in a controlled way — justified only if the pattern is strong and consistent.

Manual Challenge — Round 2 (Container Selection Game)

Problem Statement

You are given 10 containers. Each has a multiplier M and some pre-assigned inhabitants. The total pot = $10,000 \times M$. Your payout = $\text{total_pot} / (\text{inhabitants} + \text{your_share_of_all_openings})$. You can open 1 container free and a 2nd for 50,000 seashells. Choosing optimally requires predicting others' behavior.

Solution Framework

Key considerations:

- High-multiplier containers (80×, 90×) attract many players → heavily diluted payouts.
- Medium-multiplier containers (25×–50×) with few inhabitants may offer better actual EV.
- Estimate 'popularity load' for each container (how many % of players will pick it) based on its raw attractiveness.
- Container with highest multiplier gets ~8-10% of all players → huge dilution.
- The 'contrarian' approach: pick containers that look unattractive but have positive EV under rational crowding.

Optimal Strategy — P3 R2 Manual (Martin Oravec): Rank containers by:

$\text{Expected_payoff} = 10,000 \times M / (\text{inhabitants} + \text{expected_}\% \times \text{total_participants})$. Pick the single free container with highest EV. Only take the 2nd pick (cost 50k) if its expected payoff > 50,000 seashells after fee. Oravec chose Container 4 (31×, 2 inhabitants) as his free pick, skipped the paid pick. This preserved positive EV and reduced variance.

ROUND 3 — Volcanic Rock Vouchers: Options Pricing

Algorithmic Challenge

Products: Volcanic Rock (Underlying) + 5 Voucher Strikes

Volcanic Rock Vouchers are European call options on Volcanic Rock with strikes at 9,500, 9,750, 10,000, 10,250, and 10,500. They settle at expiry (end of round) with payoff = $\max(0, S - K)$. This is the most mathematically sophisticated product in all three Prosperity editions.

Black-Scholes Pricing

The Black-Scholes formula prices each voucher given the current Volcanic Rock price S , the strike K , time to expiry T , and implied volatility σ . In Prosperity, the risk-free rate r is typically set to 0.

Implied Volatility and the Volatility Smile

For each voucher, you can observe the market price and back-solve for the implied volatility σ_{implied} that makes BS price = market price. Plotting σ_{implied} against strike reveals a 'volatility smile': IV is higher for options far from at-the-money (OTM/deep ITM) than for at-the-money options. This smile is a well-known empirical pattern in options markets.

The Volatility Smile Trade: Step 1: Compute implied volatility for each of the 5 strikes. Step 2: Fit a smooth quadratic curve through the IVs as a function of log-moneyness $[\ln(K/S) / \sqrt{T}]$. Step 3: For each voucher, if market IV deviates from the smooth fit, the voucher is mispriced. Step 4: Buy underpriced vouchers (low market IV vs. smooth fit) and sell overpriced ones. Step 5: Delta-hedge your option book using Volcanic Rock.

Delta Hedging

For each voucher position, you have an options delta $\Delta = N(d1)$ from the BS formula. If you're long 10 vouchers at the 10,000 strike with $\Delta = 0.45$, your portfolio delta = 4.5 units of Volcanic Rock. To hedge, you sell 4.5 units of Volcanic Rock (holding -4.5). This neutralizes first-order price sensitivity, leaving you exposed only to volatility moves (vega).

Martin Oravec's Approach (73rd place)

Martin implemented a full IV smile pipeline: per-strike rolling IV average + quadratic smile fit, blended to stabilize noise. His signal = standardized mispricing between model price and market price, with entry at $z > 1$. He delta-hedged by summing option deltas and rebalancing when book delta moved outside a band.

Frankfurt Hedgehogs' 'Options Scalping' Approach

Frankfurt Hedgehogs called this 'Options Scalping.' Their strategy was more aggressive: beyond delta-hedging to neutralize directional risk, they actively scalped the spread between market prices and model fair values. Their dashboard showed them the IV smile curve clearly and they traded individual vouchers against their model fair value with tight bands.

jmerle's Performance by Product (P3 Round 3)

From his PnL table: Volcanic Rock +105,806, Voucher 9500 +24,176, Voucher 9750 +14,334, Voucher 10000 -11,632, Voucher 10250 -6,595, Voucher 10500 -1,485. Total from options: ~+124,600. This shows the 9500 and 9750 strikes (deep ITM and near-ATM) were the most profitable; the higher strikes (OTM) lost or broke even.

Manual Challenge — Round 3 (Two-Shot Reserve Price)

Problem Statement

Sea Turtles offer flippers with reserve prices in two bands: [160, 200] and [250, 320]. Nothing trades in the gap [200, 250]. You make two bids. Items that accept can be resold at 320. For the second bid, if your bid is below the archipelago average of all participants' second bids, your profit on those trades is scaled down by a cubic penalty.

Solution

For the first bid: maximize $(p - 160)/40 \times (320 - p)$ where $p \in [160, 200]$. Taking derivative: optimal $p = (160 + 320)/2 = 240$ — but capped at 200 since no trades occur above 200. Therefore bid 200, sweeping the entire low block at a 120-unit spread.

For the second bid: reserve prices uniform on [250, 320]. Volume-times-spread peaks at $p = (250 + 320)/2 = 285$. But the cubic penalty for being below the average means you want to cluster above 285. Martin Oravec went with 290 (slightly above the naive optimum to avoid being below average).

Optimal Bids — P3 R3 Manual: First bid: 200 (sweep all low-block fish, guaranteed 120/unit spread). Second bid: 285-295 (above naive peak of 285 due to average-aware penalty). Most top teams submitted 200/290. The penalty mechanism strongly incentivizes coordination toward higher bids in the second price.

ROUND 4 — Magnificent Macarons: Locational Arbitrage

Algorithmic Challenge

The Setup

Magnificent Macarons can be bought or sold locally and can also be converted to/from an external operator at a quoted price. The conversion involves transport fees and import/export tariffs, making the effective all-in cost higher than the nominal quote. Additionally, environmental factors (sunlight index, humidity, sugar prices) affect Macaron value.

Computing Conversion Profitability

The key calculation is the 'import basis' and 'export basis':

```
Import basis = Operator_ask + transport_fee + import_tariff
Export basis = Operator_bid - transport_fee - export_tariff
```

If $\text{local_bid} > \text{Import_basis}$: profitable to import (buy from operator, sell locally). If $\text{local_ask} < \text{Export_basis}$: profitable to export (buy locally, sell to operator). Opportunities were rare and thin — many teams found the signal too fickle to be a stable PnL driver.

The Sunlight Index Signal

Martin Oravec nearly discovered a key pattern: the sunlight index had a clear regime shift at a specific threshold value. When sunlight crossed this level, the import/export basis tended to flip predictably. Teams that discovered and correctly parameterized this threshold made significant profits; those who didn't (including Martin who ran out of time) made little from Macarons.

Sunlight Index Strategy (Near-Miss Insight from Martin Oravec): Plot: Macaron import/export basis vs. sunlight index level. Look for a critical sunlight threshold where there's a persistent flip in which direction (import vs. export) is profitable. When sunlight > threshold → trade in one direction. When sunlight < threshold → opposite direction. Backtest carefully with all-in costs included. This was the key signal that strong teams used.

RL Approach (Martin Oravec's Experiment)

Martin also tried a lightweight reinforcement learning model to learn conversion policy from features (basis, spreads, costs). He used it mainly to score and sanity-check patterns. The RL approach showed promise in-sample but was too fickle out-of-sample — a common issue with small datasets and high-dimensional state spaces in trading competitions.

Manual Challenge — Round 4 (Suitcases — Expanded Container Game)

Problem Statement

Identical structure to Round 2 containers, but now 20 suitcases, up to 3 openings, costs 0 / 50,000 / 100,000. Total per suitcase = $10,000 \times M$, shared by inhabitants + picker share.

Solution

Optimal Strategy — P3 R4 Manual (Martin Oravec): Compute $EV = 10,000 \times M / (\text{inhabitants} + \text{expected_picker_}\%)$. Rank all 20 suitcases. Free pick: best EV (e.g., C3: 73×, 4 inh → ~85,900 gross). Paid pick: 2nd best EV minus 50k fee (e.g., B4: 70×, 4 inh → ~82,400 - 50k = 32,400 net). Skip 3rd pick: no suitcase clears the 100k fee threshold. Deliberately avoid high-multiplier (90-100×) suitcases — crowd dilution is severe.

ROUND 5 — Trader IDs + Full Product Set Optimization

Algorithmic Challenge

What Changed: Named Trader Data

Like P2 Round 5, all historical trade data was de-anonymized. Teams had to find patterns in which traders (Remy, Vladimir, Olivia, Rihanna, Vinnie, new names in P3) were 'informed traders' with predictive trading behavior.

Frankfurt Hedgehogs' Approach to Trader IDs

They analyzed every trader's buy/sell patterns for each asset using their dashboard. The visualization tool (showing trader types as M=maker, S=small taker, B=big taker, I=informed trader, F=own trades) made it easy to spot informed traders visually. Their general approach: for any informed trader, if their buy orders systematically precede price increases and sell orders precede price decreases, use their trade as a leading indicator.

Martin Oravec's Refinements (Final Round)

Martin made several important strategy improvements in Round 5:

- Kelp: Replaced regression forecaster with compact z-score reversion. Keep 10-tick rolling window on mid, compute z-score, only skew quote center when $|z| > 1$. Volatility-aware spread: $\max(\min_spread, \text{rolling_std})$. Simplified but more robust.
- Rainforest Resin: Widened take band (± 2) and narrowed clear band (± 1). Added position-based skewing scaffold (set to 0, but infrastructure ready). Increased pure market-making PnL by 20%.
- Baskets: Introduced leg mean-reversion for Croissants, Jams, Djembes independently. Each leg keeps 100-tick window; when mid deviates by ~ 2 std from mean, lean the other way.
- Volcanic Rock Vouchers: Simplified to per-strike IV rolling average (window 150) + price z-score. Added steady market-maker on Volcanic Rock anchored to EMA fair ± 2 . Improved Sharpe significantly.

Manual Challenge — Round 5 (News Trading Portfolio)

Problem Statement

You are given a fictional newspaper with stories about fictional companies. Based only on these stories, allocate 1,000,000 seashells across long/short positions. You profit proportionally to the actual move $\times$ your allocation. A fee structure penalizes extremely large positions in any single stock.

Martin Oravec's Portfolio (Scored 100,469 seashells, best $\sim 200,000$)

Stock	Direction	Allocation	Rationale
Quantum Coffee	Strong Sell	25%	Health/regulatory overhang — major risk event
Solar Panels	Strong Sell	10%	Tax hike triples ownership cost — policy shock
VR Monocle	Strong Buy	8%	MAU surge + session length up — momentum signal
Moonshine	Weak Buy	5%	Credible 'next-launch' signal
Striped Shirts	Medium Buy	5%	Season-ahead trend call
Cacti Needle	Medium Sell	5%	Public blame for derailment — litigation risk
Red Flags	Medium Sell	4%	Operations damaged — avoid catching falling knife
Ranch Sauce	Weak Buy	2%	Takeover premium + deal upside
Haystacks	Weak Buy	2%	Niche revival chatter — small speculative bet

Key News Trading Lessons

- Not all news is tradeable — only act on clear, unambiguous catalysts.
- Regulatory shocks and product liability events are strong directional signals (sell).
- User growth/engagement metrics are strong signals (buy) for tech-like products.
- Diversify: the fee structure penalizes concentration, so spread across 6-9 positions.
- Strong signals justify up to 25% allocation; weak signals should be 2-5% at most.
- Teams earning 200k vs. 100k likely had better information about how the actual prices moved — or had prior knowledge from P2's similar challenge.

PART IV: CROSS-COMPETITION PATTERNS & META-INSIGHTS

Product Recurrence Table

Prosperity consistently re-uses product archetypes. Understanding each archetype means you can immediately recognize and apply the known strategy:

Archetype	P1 Name	P2 Name	P3 Name	Core Strategy
Stable Asset	Pearls	Amethysts	Rainforest Resin	Market-make at constant fair value 10,000
Random Walk	Bananas	Starfruit	Kelp	Popular mid / Wall Mid market-making
Volatile/Signal	Diving Gear	Orchids	Squid Ink	External signal filtering or arbitrage
Seasonal Curve	Berries	—	—	Buy trough, sell peak at known timestamps
ETF Basket	Picnic Basket (4+2+1)	Gift Basket (4×Choc+6×Str+1×Rose)	Picnic Basket 1&2	Spread z-score arb vs. synthetic fair
Options	—	Coconut Coupons	Volcanic Rock Vouchers (5 strikes)	Black-Scholes IV arbitrage + delta hedge
Location Arb	—	Orchids	Magnificent Macarons	Import/export basis + environmental signal
Pair Trade	Coco/Pina	—	—	Ratio spread z-score between related products

Manual Round Recurrence Table

Manual Type	P1	P2	P3
Fish Gear Auction (two-price)	R1	R1, R4	—
Estimate Value / Simulation	R2, R3	—	R3
Currency Chain / Graph Arb	—	R2	R1
Container/Suitcase EV Game	—	R3	R2, R4
Portfolio Stock Allocation (News)	—	R5	R5
Capital Allocation Formula	R4	—	—

Algorithmic Trading Strategy Blueprints

Blueprint 1: Market Making on Stable Assets

When to use: Stable-price product (constant true value, e.g., 10,000). Bots occasionally trade irrationally.

Implementation:

- Identify the true/fair value (given in problem or inferred from data — look for a round number where most trades cluster).
- Each tick: first take (buy asks below fair, sell bids above fair); then make (post buy at fair-2, sell at fair+2).
- Inventory management: if position > 80% of limit, narrow one side of the quote; at limit, close at fair.
- Tune the quote distance (1-3 units from fair) using backtesting.

Blueprint 2: Market Making on Random Walk Assets

When to use: Asset price drifts slowly (random walk) with no predictable direction. Cannot forecast next price reliably.

Implementation:

- Compute fair value each tick as Wall Mid or Popular Mid (midpoint between volume-weighted bid and ask).
- Quote ± 1 to ± 2 from current fair value. Width depends on typical spread; wider if spread is large.
- Manage inventory: add a soft skew when position > 50% of limit, hard close at limit.
- Update fair value every tick (it changes!), so always quote around the freshest estimate.

Blueprint 3: ETF / Basket Statistical Arbitrage

When to use: A basket product whose fair value = sum of component prices $\times$ quantities. Basket price deviates from synthetic fair.

Implementation:

- Every tick: compute Synthetic = sum(qty_i $\times$ mid_i) + known_premium.
- Spread = Basket_mid - Synthetic. Compute rolling z-score over window W (typically 50–200 ticks).
- When $z > \text{threshold}$ (e.g., 1.5): SELL basket (it's overpriced). When $z < -\text{threshold}$: BUY basket.
- Optional hedge: trade components in the opposite direction to delta-hedge. Risk: component limits.
- Exit when z returns toward 0, or use a separate exit threshold.
- Grid-search the entry threshold on historical data. Beware overfitting — use walk-forward validation.

Blueprint 4: Black-Scholes Options Trading

When to use: European call option-like product with known underlying, strike, expiry. Market price deviates from BS fair value.

Implementation:

```
from scipy.stats import norm import numpy as np def bs_call_price(S, K, T, sigma, r=0):
if T <= 0: return max(0, S - K) d1 = (np.log(S/K) + (r + 0.5*sigma**2)*T) /
(sigma*np.sqrt(T)) d2 = d1 - sigma*np.sqrt(T) return S * norm.cdf(d1) - K *
np.exp(-r*T) * norm.cdf(d2) def bs_delta(S, K, T, sigma, r=0): if T <= 0: return 1.0
if S > K else 0.0 d1 = (np.log(S/K) + (r + 0.5*sigma**2)*T) / (sigma*np.sqrt(T))
return norm.cdf(d1) def get_implied_vol(market_price, S, K, T, r=0, tol=1e-6): #
```

```

Binary search for sigma such that bs_call_price == market_price      lo, hi = 0.01, 5.0
for _ in range(100):          mid = (lo + hi) / 2          price = bs_call_price(S, K, T,
mid, r)          if price < market_price: lo = mid          else: hi = mid          if hi -
lo < tol: break          return (lo + hi) / 2

```

- Compute BS price each tick. If market_price > BS_price by threshold: SELL the option. If market_price < BS_price: BUY.
- Delta-hedge: after each option trade, offset by trading $-\Delta$ units of the underlying.
- For multiple strikes (P3): compute IV per strike, fit a smooth smile, trade those most deviant from the smile.
- Key parameter to calibrate: σ (implied vol). Use observed market prices to back out σ , track rolling average.

Blueprint 5: Informed Trader Signal Exploitation

When to use: Round 5 of any Prosperity edition (trader IDs revealed). Some traders have consistent directional accuracy.

Implementation:

- For each trader ID in trade history, compute: win rate = fraction of their trades where price subsequently moved in their favor.
- If win rate > 55-60% consistently across all days: this trader is informed.
- Strategy: follow their trades. If they buy, you buy (or go long if you're flat). If they sell, you sell/go short.
- Limit exposure: only trade when informed trader signal appears. Keep position flat otherwise.
- Use jmerle's grid-search approach: test all combinations of (trader, product, lag) for predictive power.

PART V: PREPARATION GUIDE FOR IMC PROSPERITY 4

Phase 1: Before the Competition Starts (NOW)

1. Master the Core Strategies

You now understand all the strategies used in the past three competitions. Before P4 starts, implement and test each one from scratch:

- Market-making on a stable asset (Pearls/Amethysts/Resin equivalent)
- Market-making on a random walk asset (Bananas/Starfruit/Kelp equivalent)
- ETF basket stat-arb (Picnic Basket / Gift Basket equivalent)
- Black-Scholes options pricing and trading
- Locational arbitrage with cost calculation
- Informed trader signal detection

2. Build Your Infrastructure

Infrastructure is as important as strategy. Build these before the competition:

- Backtester: Download jmerle's open-source backtester (jmerle/imc-prosperity-3-backtester on GitHub) and learn to use it. Customize it to log detailed PnL per product per tick.
- Visualizer: Use jmerle's visualizer to see order book dynamics. Understanding visual patterns in the order book is essential.
- Dashboard: Consider building a custom dashboard like Frankfurt Hedgehogs did — scatter plot of bids/asks over time, PnL panel, position panel.
- Logging framework: Implement structured logging in your algorithm (use traderData to pass logs back). You want to see: fair value estimate, current spread, fills, position, PnL.

3. Study P1/P2/P3 Data

The historical data from all three competitions is publicly available in the GitHub repositories. Download it and run analyses:

- Plot price series for every product to understand their character (stable, random walk, trending, seasonal).
- Look for cross-year similarities — if P4 Coconut data looks like P1/P2/P3 Coconut data, you may already know the future.
- Compute rolling correlations between products to identify potential pair-trading opportunities.
- Study the order book depth patterns to understand where 'walls' typically form.

4. Learn the Key Mathematical Concepts

Study these if you haven't already:

- Black-Scholes formula: implement it, understand all inputs, know how to compute Greeks (delta, vega, gamma).
- Z-score and rolling statistics: rolling_mean , rolling_std , $\text{z-score} = (x - \text{mean}) / \text{std}$.
- Linear regression: for price prediction (bananas/starfruit approach). Simple OLS in Python via `numpy.polyfit`.
- Expected Value (EV) calculation: for manual rounds — $E[\text{profit}] = \sum(P(\text{outcome}) \times \text{profit}(\text{outcome}))$.
- Nash Equilibrium basics: for container/suitcase manual rounds — game-theoretic reasoning about others' optimal strategies.

Phase 2: During Each Round (Execution)

Round Start Protocol

- Day 1-6 (first few hours): Analyze the new data. Plot everything. Don't rush to submit — a wrong algorithm costs you.
- Identify which product archetype each new product belongs to. Match to a known Blueprint (above).
- If an external signal exists (sunlight, dolphin sightings), immediately plot signal vs. price. Look for threshold effects.
- Run the provided historical data through jmerle's backtester with your initial strategy idea.

Algorithm Development Process

- Start simple: implement the most obvious strategy first (e.g., market-making if it's a stable product).
- Test thoroughly on historical data before submitting.
- Add complexity one feature at a time. Test again after each addition.
- When adding a new product to your algorithm, run the full backtester on all products together (interactions matter).
- Grid-search key parameters (entry threshold, quote distance, lookback window) on the provided data.
- Validate your best parameters are robust: test on different days, check performance is consistent.

The 'Do No Harm' Principle

Critical Risk Management: A strategy that loses money is worse than no strategy at all. If you cannot find a consistently profitable strategy for a product in backtesting, DO NOT TRADE IT. Products with negative PnL drag down your entire ranking. It's better to earn 0 on a hard product than -50k.

Manual Round Protocol

- Fish Auction (if it reappears): Run Monte Carlo simulation with 1M+ samples. The answer should converge to 952/978 (or a similar variant).
- Container/Suitcase EV games: Write a Python script to compute EV for every container/suitcase under different crowd assumptions. Be contrarian on high-multiplier options — they're always overcrowded.
- Currency chain: Write BFS to find the optimal conversion sequence. Always check for arbitrage cycles.
- News portfolio: Identify 5-8 clear catalysts. Size by conviction (25% max, 2% min). Diversify. Check the fee structure.

Phase 3: Cross-Round Refinement

After Each Round

- Download your round log immediately. Visualize your fills, PnL, and position over time.
- Look for patterns: Were there times your algorithm had a large position it couldn't exit? Were there obvious mispricings you missed?
- Compare your PnL per product against what top teams report in Discord — identify which products you're leaving money on the table.

- Read the Discord carefully: community hints are invaluable. Being active and helpful (like jmerle was with his tools) also generates goodwill and hints in return.

The 'Escalating Strategy' Framework

A framework for deciding how complex your strategy should be for each product:

- Level 1 (Default): Simple market-making at Wall Mid. Start here for EVERY product.
- Level 2 (If stable): Add aggressive taking of favorable prices. Tune quote distances.
- Level 3 (If ETF relationship exists): Add z-score spread trading. Test component hedging.
- Level 4 (If options exist): Implement Black-Scholes. Add IV smile if multiple strikes.
- Level 5 (If trader IDs revealed): Add informed trader signal overlay.
- Level 6 (If external signals exist): Add threshold-based regime detection.

Expected Products in Prosperity 4 (Speculation)

Based on the pattern of the first three competitions, here's what to expect and prepare for:

Round	Likely Product Type	Preparation
Tutorial/R1	Stable asset (=10,000) + Random walk	Market-making blueprints ready
Tutorial/R1	Third 'polarizing' asset (like Squid Ink)	Test stat-arb, volatility strategies
Round 2	ETF baskets (1-2 baskets + components)	Spread z-score arb framework ready
Round 3	Options with multiple strikes	Black-Scholes + IV smile code ready
Round 4	Locational arbitrage + environmental signal	Import/export basis + threshold detection
Round 5	All above + trader IDs	Informed trader signal detection ready

Python Tools & Libraries for Prosperity

Library	Use Case
numpy	Rolling statistics, linear algebra, array math
scipy.stats	Normal CDF for Black-Scholes (norm.cdf)
scipy.optimize	Implied volatility root-finding
collections.deque	Efficient rolling window of fixed max length
json	Serialize/deserialize traderData for state persistence
itertools	Grid search over parameter combinations

Common Mistakes to Avoid

- Submitting an untested algorithm: ALWAYS backtest before submitting. Use jmerle's backtester.
- Overfitting to 3 days of historical data: Test on each day separately. If parameters that work on Day 1 fail on Day 2, they will fail in production.
- Not managing inventory: An algorithm that never manages its position will hit limits and become unable to trade profitable opportunities. Implement soft/hard liquidation from Day 1.
- Trading every product with every strategy: It's better to do 3 products extremely well than 10 products mediocrely. Focus.
- Ignoring the manual rounds: Even a 50k difference in manual PnL can move you 50+ positions. Spend the necessary time on each manual challenge.

- Not reading Discord: The community shares critical insights (sometimes even specific strategy hints). Being on Discord during the competition is non-negotiable.
- Complex models with limited data: With only 3 days of data (roughly 30,000 ticks per round), complex ML models will overfit. Keep models simple and robust.

The Single Most Important Mindset Shift

"Be rigorous about not overfitting, and avoid complicated models due to the lack of test data."

— *Stanford Cardinal (2nd Place, Prosperity 1)*

This quote encapsulates the entire preparation philosophy. Prosperity rewards those who correctly identify the market mechanism, implement a clean strategy, and resist the temptation to over-engineer. The teams that finished 2nd in P1 and P3 both made this principle central to their approach. Simple, correct, robust beats complex, clever, fragile — every time.

QUICK REFERENCE SUMMARY

All Manual Challenges — Quick Reference

Competition	Round	Challenge Type	Optimal Answer / Approach
P1	R1	Fish gear two-bid auction	L=952, H=978 (calculus / Monte Carlo)
P1	R2	Estimate median of bottom 50% fish prices	Simulation; ~9,685
P1	R3	Guess target price	~9,969; use estimation / eyeballing
P1	R4	Capital allocation: $75xy-90y^2$ formula	$y^* = 2.5x/6$ per stock; invest proportionally to conviction
P2	R1	Fish gear two-bid auction (same)	L=952, H=978 (same formula)
P2	R2	Max conversion chain (6 trades)	BFS on exchange rate graph; ~13.9% profit
P2	R3	Treasure map expedition	Game theory + community data; 2 expeditions optimal
P2	R4	Fish gear auction (average-aware)	L=960, H=980 (cluster above expected average)
P2	R5	News portfolio (Iceberg news)	Conviction sizing; 6-8 positions; 10-25% each
P3	R1	FX arbitrage (max 5 trades)	BFS on exchange graph; ~8.9% profit
P3	R2	Container EV game (10 containers)	Contrarian: mid-multiplier, low-inhabitant; skip 2nd if EV < 50k
P3	R3	Two-shot reserve price (cubic penalty)	First bid=200, Second bid=290
P3	R4	Suitcase EV game (20 suitcases)	Contrarian: best EV suitcase free, 2nd if EV>50k; skip 3rd
P3	R5	News portfolio	Same as P2 R5; strong signals 25%, weak 2-5%

All Algorithmic Challenges — Quick Reference

Competition	Round	Product	Best Strategy
P1	R1	Pearls	Market-make at 10,000; take any price above/below
P1	R1	Bananas	Linear regression on last N ticks; market-make + take
P1	R2	Coconuts / Pina Coladas	Pair trading: z-score on spread (Pina - $15/8 \times$ Coco)
P1	R3	Berries	Hardcode buy at t=350k, sell at t=500k
P1	R3	Diving Gear	Trade only on dolphin sighting $\Delta \geq 5$
P1	R4	Picnic Basket	Z-score arb: spread vs. components+375 premium
P1	R5	Ukulele	Follow Olivia (informed trader signal)

P2	R1	Amethysts	Market-make at 10,000
P2	R1	Starfruit	Popular mid market-making + soft/hard liquidation
P2	R2	Orchids	Short-sell to limit + convert to flat; repeat same tick
P2	R3	Gift Basket	Z-score spread arb vs. $(4 \times \text{Choc} + 6 \times \text{Str} + 1 \times \text{Rose})$
P2	R4	Coconut Coupons	Black-Scholes pricing; trade when market deviates from model
P2	R5	Chocolate/Roses	Follow Vladimir/Remy (Choc), Rihanna/Vinnie (Roses)
P3	R1	Rainforest Resin	Market-make at 10,000; Wall Mid for take/make
P3	R1	Kelp	Wall Mid market-making; skip predictive models
P3	R1	Squid Ink	Only trade if backtest is robust; skip if unclear
P3	R2	Picnic Basket 1&2	Z-score arb on both basket spreads; two-way stat-arb
P3	R3	Volcanic Rock Vouchers	Black-Scholes IV smile fit; delta-hedge with Volcanic Rock
P3	R4	Magnificent Macarons	Sunlight index threshold detection; import/export basis
P3	R5	All products	Add informed trader signals; refine all existing strategies

Good luck in IMC Prosperity 4! May your PnL be positive and your bugs be few.